Rapport

Alexandre Lou-Poueyou
Aymeric Cassard
Ludovic Andrieu
Jean Mayol
Matheo Petry
Valentin Cazet

23 janvier 2025

Résumé

Ce document est le rapport du projet Scrabble en Java.

Table des matières

1 Le contexte et l'existant du projet **2**

2 Une explication détaillée du sujet **2**
2.1 Règles du jeu 2
2.2 Structures de données 3
2.3 Algorithmes spécifiques 4

3 La liste des besoins visés **4**

4 Un agenda prévisionnel des livraisons des besoins **4**

5 Architecture du projet **8**
5.1 Modules du projet 8
5.2 Dépendances du projet 9

1 Le contexte et l'existant du projet

2 Une explication détaillée du sujet

2.1 Règles du jeu

Le Scrabble est un jeu de société qui consiste à former des mots sur une grille carrée de 15x15 cases. Les 100 lettres utilisées ont une valeur de point choisie selon la fréquence du mot dans la langue. 2 lettres 'Joker' supplémentaires sont présentes, elles permettent au joueur de placer n'importe quelle lettre, mais n'ajoutent aucun point au mot. Chaque joueur dispose de lettres tirées aléatoirement qu'il place sur la grille pour marquer des points.
La grille contient des cases bonus :

— Lettre double : double la valeur de la lettre posée.
— Lettre triple : triple la valeur de la lettre posée.
— Mot double : double la valeur du mot posé.
— Mot triple : triple la valeur du mot posé.

Les multiplicateurs de mots se combinent dans le cas où le mot posé touche plusieurs cases.

Le mot doit exister dans le dictionnaire courant du programme pour pouvoir être posé. Les éventuels mots adjacents au mot posé rapportent des points.

Chaque joueur dispose de 7 lettres dans sa main. Si un joueur pose toutes ses lettres en un tour, alors s'applique un bonus de 50 points.
Les actions que peut entreprendre le joueur lors de son tour sont :

— Poser un mot.
— Échanger des lettres de la pioche.
— Passer son tour.

Les conditions de fin de partie sont les suivantes :

— **Reliquat épuisé** : Un joueur a épuisé toutes ses lettres et la pioche est vide.
— **Blocage** : Les joueurs ne trouvent plus de mots a jouer et il reste moins de 7 lettres dans la pioche. Les valeurs des lettres de leur mains sont déduites de leur score final.
— **Temps épuisé** : Les deux joueurs n'ont plus de temps (variante Blitz uniquement).

Le joueur avec le score le plus élevé est le gagnant.

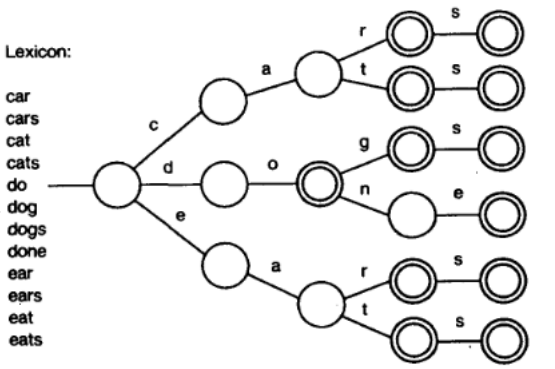
Variantes

— **Super-scrabble** : La grille fait 21x21.
— **Blitz** : Chaque joueur dispose d'un temps imparti, qui s'écoule lors de son tour. Si un joueur n'a plus de temps, il ne peut plus jouer.

2.2 Structures de données

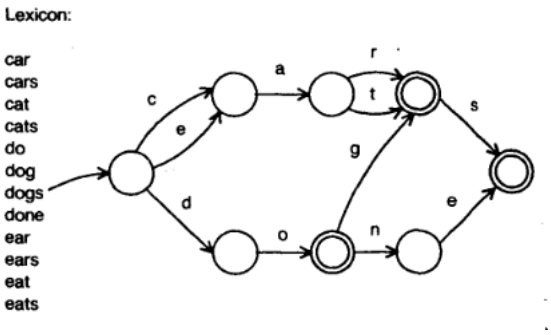
Structures de données simples
L'utilisation de Java dans le projet permet d'éviter d'implémenter par nous-mêmes les structures de données de base du projet. Par exemple, la pioche est représentée par une HashTable qui associe chaque lettre à un compteur, représentant le nombre de lettres restantes dans la pioche.

Représentation du dictionnaire
Une manière commune de représentation de lexique est un arbre pour lequel chaque arête est une lettre du mot, et certains nœuds sont marqués comme finaux, indiquant que le chemin jusqu'à ce noeud constitue un mot. Cette structure est communément appelée un trie.



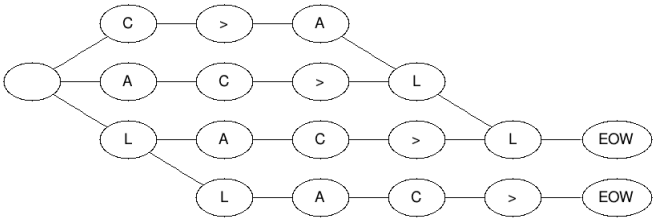
Le trie n'est pas utilisé dans notre projet mais aide à visualiser le fonctionnement des structures de données suivantes.

Le **DAWG**(Directed Acyclic Word-Graph) nous permet de représenter le dictionnaire de validité des mots en mémoire. Il s'agit d'un automate fini pour lequel le langage est l'ensemble des mots du dictionnaire, et les transitions sont les lettres que l'on ajoute pour former le mot. Enfin, les états finaux sont les différents mots du lexique.



Cette structure de données est équivalente au trie en termes de fonctionnalités, mais prend moins de place en mémoire.

Le **GADDAG** est une autre manière de représenter le lexique. Dans cette structure, chaque mot est représenté à partir de chacune de ses lettres intermédiaires.



Cette structure est une optimisation du DAWG en terme de calcul des mots possibles tout en respectant les contraintes du scrabble. Le GADDAG est donc utilisé dans les algorithmes de recherche de meilleur coup par IA. Cependant, cette structure prend plus de place en mémoire que le DAWG.

2.3 Algorithmes spécifiques

Nous utilisons deux algorithmes de recherche dans la partie intelligence artificielle du projet.

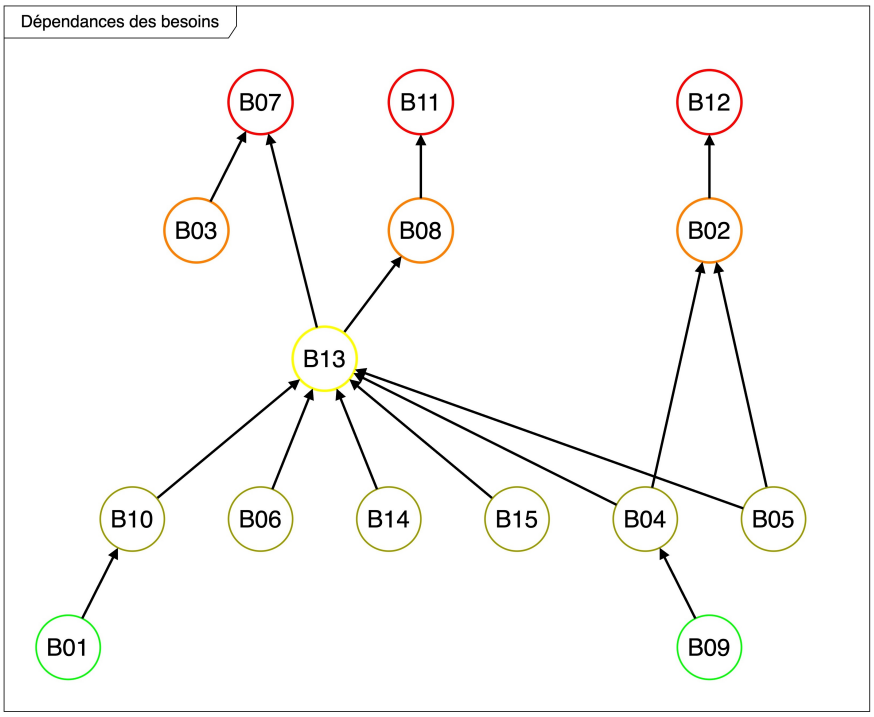
— **MiniMax**
— **Monte-Carlo**

3 La liste des besoins visés

4 Un agenda prévisionnel des livraisons des besoins

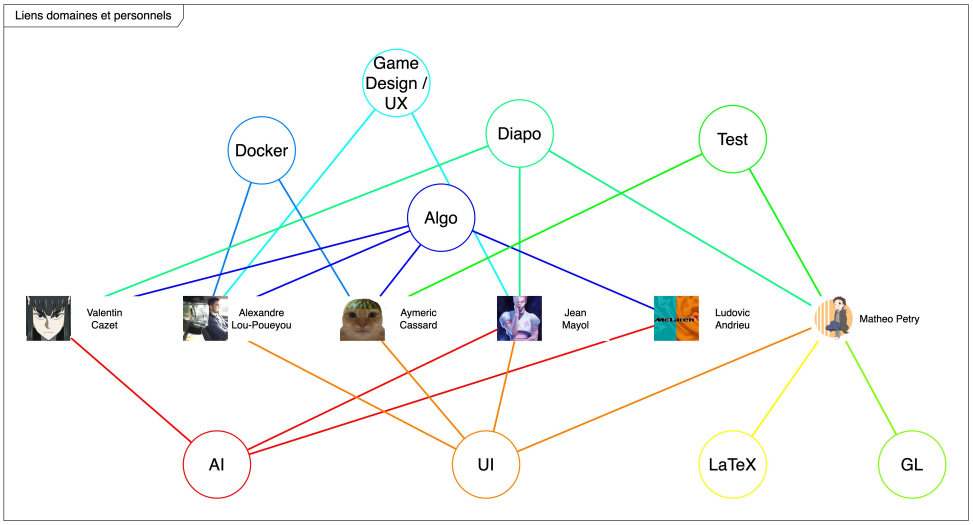
Pour établir l'agenda prévisionnel, nous allons nous appuyer sur le graphe des dépendances entre les besoins. Ce graphe nous permet d'identifier que certains

besoins nécessitent d'autres besoins déjà implémentés. Par exemple, le besoin B01 dépend du besoin B10.

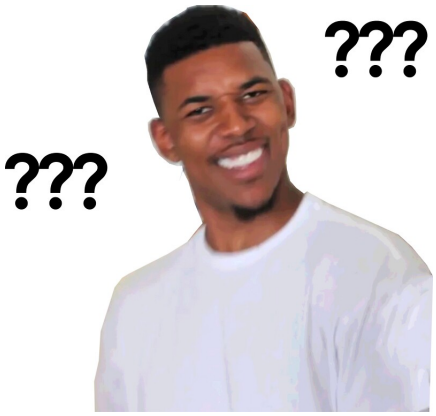


Nous en concluons donc que nous devons d'abord prioriser l'implémentation des besoins B07, B11 et B12.

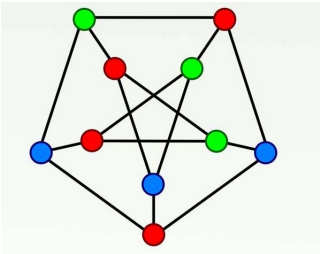
Ensuite, nous avons réalisé un sondage auprès des membres de l'équipe afin de déterminer leurs préférences en matière de développement. Certains souhaitent se concentrer sur l'implémentation de l'AI et des algorithmes. Il est également important de noter que certains ne souhaitent pas s'occuper des phases de test, car ils ne sont pas dans le parcours GL. Il est donc préférable de confier cette tâche aux personnes spécialisées dans ce domaine.



Pour établir notre agenda prévisionnel, nous devons donc combiner le graphe des dépendances avec les préférences de développement des membres de l'équipe. Mais comment faire ?



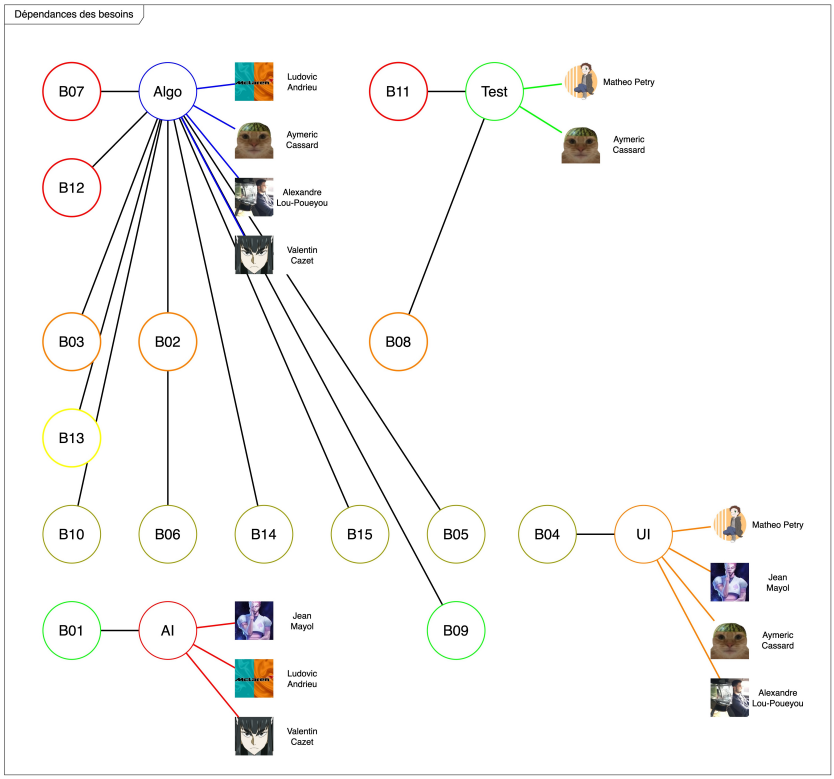
Nous pourrions réduire ce problème à un problème de coloration de graphe. Cependant, nous avons décidé de ne pas suivre cette approche, car cela nécessiterait trop de temps pour le codage, d'implémentation et les tests. Nous avons estimé que ce développement prendrait au moins une heure, alors que le faire manuellement ne prend que 5 minutes.



Nous avons donc choisi de combiner ces deux graphes manuellement, en faisant en sorte qu'un membre de l'équipe prenne en charge les dépendances et les associe ensemble.

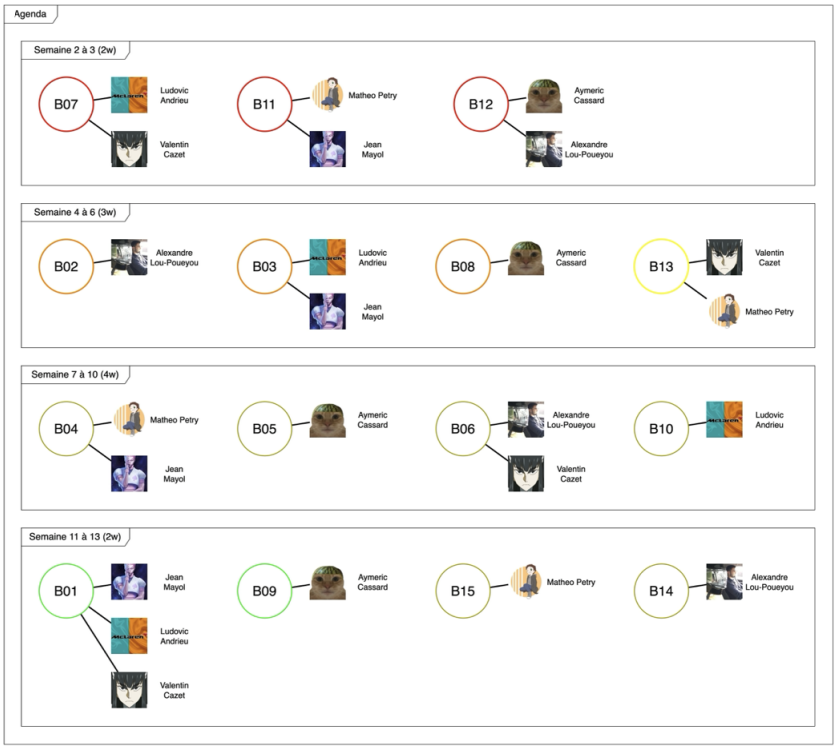


Nous obtenons ainsi un graphe qui combine les besoins, leur ordre de développement, les domaines d'intervention dans un projet, ainsi que les membres de l'équipe.



Il ne reste plus qu'une étape pour générer l'agenda prévisionnel : pour chaque besoin, il faut choisir une personne responsable de son développement.

Nous avons donc établi notre agenda prévisionnel.



Par exemple, de la Semaine 2 à la Semaine 3, soit 2 semaines de travail, le besoin B07 sera développé par Ludovic et Valentin. Il en va de même pour le besoin B11, qui sera pris en charge par Mathéo et Jean, etc.

5 Architecture du projet

5.1 Modules du projet

Nous avons choisi une architecture de style MVC pour organiser les modules du projet. Ce motif d'architecture a été choisi car il pose des lignes directrices simples et comprises par tous les membres vis à vis de la question du placement des différents modules. De plus, il se prête bien a la mise en place de la partie graphique. Conformément a ce modèle, nos modules sont donc découpés en trois parties.

Modèle

Le modèle contient les différentes structures de données, et autres éléments logiques relatifs aux contraintes du jeu. Les éléments de cette couche sont : Game, AI.
Vue
La vue est constituée de tous les éléments visibles par l'utilisateur. Ici, elle est découpée en deux grands axes. La partie CLI, et la partie GUI.
Controlleur
Cette couche de l'architecture traite les interactions avec l'utilisateur. C'est donc cette partie qui s'occupe d'actualiser le modèle selon les différentes actions de l'utilisateur.

5.2 Dépendances du projet

Le projet est codé en Java version 17.0.13, et est compilé a l'aide du système de build Maven. Afin de respecter les spécifications du projet, un certain nombres de dépendances sont ajoutées :

- **JUnit** : Framework de test, utilisé pour les différents scénarios de test du code.
- **Jacoco** : Librairie de couverture de code, utilisée en conjonction avec JUnit pour certifier le pourcentage de code source testé.
- **Commons-cli** : API d'analyse d'options de commandes. Utilisé pour interpreter les options passées en ligne de commande.
- **JavaFx** : Framework d'interface utilisateur, utilisé pour la partie interface graphique.
- **Checkstyle** : Outil d'analyse de code permettant de vérifier la conformité du formattage avec le style de codage de Google.